BSc (Hons) in **Computer Science**

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure (P) – Defines how success is evaluated (score, rewards, penalties).
Environment (E) – The external world in which the agent operates (grid, walls, food, opponents).
Actuators (A) – The mechanisms used by the agent to perform actions (Up, Down, Left, Right).
Sensors (S) – The mechanisms used by the agent to perceive the environment (get_percept() data).

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width → Grid width
self.height → Grid height
self.walls → Locations that block movement
self.food_positions → Locations of food rewards
self.opponents → Other agents in the environment
self.agent_pos → Current position of the agent
self.collision → Collision state
self.steps → Number of actions taken

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent environment. because The environment contains self.opponents = []. The opponents act independently and can move around the grid. Since multiple entities are making decisions and influencing the environment, this is classified as a multi-agent environment.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of observations received by an agent through its sensors over time.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensor (S) component

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The environment is considered Partially Observable because the agent does not have access to the complete state of the environment at all times. Although the get_percept() method provides information such as the agent's current position, opponent positions, food locations, and score, it does not provide all possible information about the environment, such as hidden hazards or future opponent actions. Since the agent must make decisions using incomplete information, the environment is classified as partially observable.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure (P)

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

An intelligent agent should be evaluated by the effect of its actions on the environment, not by how much internal computation it performs.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

If toxic traps exist but are hidden from sensors, the environment becomes more Partially observable.The agent cannot know where traps are located and which positions are dangerous ,Therefore, it must make decisions under uncertainty.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The new sensor improves rational decision-making because the agent receives additional information before acting.

Example:

Without toxin detection:

Move Right → Unknown result

With toxin detection:

smells_toxin = True
→ Avoid that location
→ Choose safer action

The agent can maximize expected utility by:

Avoiding penalties
Preserving score
Choosing safer paths

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World exploit example demonstrated the danger of using an incorrect performance measure in an intelligent agent. In the example, if the agent was rewarded only for cleaning dirt, it could exploit the metric by repeatedly cleaning the same already-clean location instead of actually improving the environment. This showed that a performance measure must correctly represent the real objective of the agent. Similarly, in this practical, adding a severe penalty for stepping on toxic traps prevents the agent from exploiting the scoring system and encourages it to make safer decisions that maximize its overall performance.

Finalize and Lock Lab 01 Analytical Evaluation

FACULTY OF COMPUTING